

Branch-Aware Logical Redo Logging for Relational Databases

Derek Yang
University of Michigan
Ann Arbor, USA
dereky@umich.edu

Alex Bartolozzi
University of Michigan
Ann Arbor, USA
alexbart@umich.edu

Conner Rose
University of Michigan
Ann Arbor, USA
cnrose@umich.edu

1 Introduction

Many data analysis and projection workflows are increasingly resembling software development workflows in which independent versions of datasets support “what-if” scenario analysis. In software engineering, version control systems such as Git allow developers to create lightweight branches, experiment in isolation, and merge results back into a shared mainline. No comparable primitive exists within relational database systems. Without built-in support, users must resort to full table duplication, external data-lake versioning systems, or ad-hoc snapshotting mechanisms—all of which introduce storage overhead, operational complexity, and limited query awareness.

The problem we address is: how can we support branchable relational tables while avoiding full database duplication? We want users to be able to create, query, and merge branches as if each branch were its own independent snapshot, while keeping storage costs proportional only to the changes made on that branch.

We present a branch-aware logical redo logging framework implemented as a PostgreSQL extension. Instead of copying entire tables to create a branch, we maintain a per-branch *delta table* that records logical row-level changes (inserts, updates, deletes). At query time, a SQL view overlays the delta onto the parent table, reconstructing the branch state without data duplication. Write interception is handled transparently through PostgreSQL’s `INSTEAD OF` trigger mechanism, so users interact with branched tables using ordinary SQL.

We arrived at this design through an iterative process, exploring three distinct architectures—temporary table materialization per query, eager materialized copies per branch, and view-based delta overlay—each addressing limitations discovered in its predecessor. We evaluate all three approaches using the TPC-H benchmark suite at scale factor 1, focusing on the read overhead introduced by the branch abstraction layer over native PostgreSQL query execution.

Our results show that the view-based delta overlay achieves near-zero overhead (median 1.02×) for the majority of TPC-H queries, with a small number of queries exhibiting higher overhead due to fundamental limitations in how PostgreSQL’s query planner handles correlated subqueries through views. We analyze these limitations and discuss potential hybrid strategies that could address them.

2 Methods

All three architectures share a common foundation. The extension maintains a metadata table, `branch.branches`, that records each branch’s name, parent, base table, and associated delta table. A session-level GUC variable (`branch.active_branch`) tracks which branch the user is currently on, and `switch_branch` updates the PostgreSQL `search_path` so that unqualified table references resolve to the active branch’s schema. This allows users to write ordinary SQL—`SELECT * FROM users`, `INSERT INTO users`, etc.—without any branch-specific syntax.

Each branch has a per-branch *work schema* (`branch_work_<name>`) that contains either a table or a view with the same name as the base table, enabling transparent resolution via `search_path`. Each branch also has a *delta table* in the branch schema that stores a log of row-level changes. Every delta row contains an `_op` column (I/U/D for insert, update, delete), a monotonically increasing `_seq` sequence number, and the full row payload. To resolve the latest state of any primary key, we use `DISTINCT ON (pk_cols) ORDER BY pk_cols, _seq DESC`, which selects the most recent operation per key.

Branches also support two lifecycle operations: `apply_branch` replays the delta log into the parent table (analogous to a Git merge), and `rollback_branch` discards all changes by truncating the delta table.

2.1 Temporary Table Per Query

Our first architecture faithfully implemented the “log is the database” philosophy from our proposal. Each read from a branched table triggered the following sequence: (1) create a temporary table as a full copy of the base table, (2) replay all delta log entries on top of it to reconstruct the branch state, (3) execute the user’s query against the temporary table, and (4) drop the temporary table.

This approach had the advantage of simplicity and correctness—the reconstructed table was a first-class PostgreSQL table with full index support. However, it required a complete table copy *on every query*, and was effectively a proof of concept for our overall interface design. For the TPC-H `lineitem` table at scale factor 1 (approximately 6 million rows), this resulted in roughly 1000× overhead compared to native execution. The cost was dominated by the copy and

drop cycle, not the delta replay itself. This architecture was clearly unsuitable for any practical workload.

2.2 Materialized Copy Per Branch

To eliminate the per-query copy overhead, our second architecture created a full materialized copy of the base table at branch creation time. Each branch’s work schema contained a real table populated with

```
CREATE TABLE ... AS SELECT * FROM parent. An
AFTER ROW trigger on the materialized table intercepted
writes and appended corresponding entries to the delta table,
maintaining the change log for apply and rollback operations.
Reads from the branch were direct table reads with
no overlay logic—identical in performance to native PostgreSQL.
```

Importantly, the materialized copy was only created for the currently active branch. At any given time, the system maintained at most two full copies of the data: the base table and the working copy for the active branch. Switching branches would drop the current working copy and materialize a new one by replaying the target branch’s delta log onto a fresh copy of the base table.

This approach delivered native read performance, since queries hit a real table with full index and parallel query support. However, it introduced significant cost at branch creation and switching time, both of which were proportional to the size of the base table (approximately 15 seconds for `lineitem` at `SF=1`). While storage was bounded to at most two full table copies rather than one per branch, the latency of branch operations made interactive workflows impractical for large tables.

2.3 View-Based Delta Overlay

Our final architecture eliminates data copying entirely. Instead of a materialized table, each branch’s work schema contains a SQL *view* that overlays the delta table onto the parent’s data at query time. Branch creation is near-instant regardless of base table size—it only creates an empty delta table, a view definition, and trigger functions.

The view is defined as a `LEFT JOIN` anti-join between the parent data and the delta, unioned with the insert and update entries from the delta:

```
SELECT base.cols FROM parent base
LEFT JOIN (
  SELECT DISTINCT ON (pk) pk
  FROM delta ORDER BY pk, _seq DESC
) d ON base.pk = d.pk
WHERE d.pk IS NULL
UNION ALL
SELECT cols FROM (
  SELECT DISTINCT ON (pk) _op, cols
  FROM delta ORDER BY pk, _seq DESC
) d2 WHERE d2._op IN ('I','U')
```

The first half selects all parent rows whose primary keys do *not* appear in the delta (i.e., unchanged rows). The second half selects the latest version of each inserted or updated row from the delta. Deleted rows are excluded because they appear in the delta with `_op='D'`, causing them to be filtered out of both halves.

Write interception uses PostgreSQL’s `INSTEAD OF` trigger mechanism on the view. When a user executes an `INSERT`, `UPDATE`, or `DELETE` against the view, the trigger function appends the corresponding delta entry rather than modifying the base table. This makes writes transparent—the user writes ordinary SQL and the branch layer captures the changes automatically.

An important design decision was the choice of `LEFT JOIN` over `NOT IN` for the anti-join. Our initial implementation used `NOT IN (SELECT pk FROM latest)`, which prevented PostgreSQL’s parallel query executor from engaging—the `NOT IN` subplan is evaluated per-row and cannot be parallelized. Switching to a `LEFT JOIN ... WHERE d.pk IS NULL` pattern allows PostgreSQL to use a Hash Anti Join, which supports parallel sequential scans. This single change reduced overhead on TPC-H Q6 from 2.75× to 1.02×.

3 Performance Results

3.1 Benchmark Setup

We evaluate the view-based delta overlay architecture using the TPC-H benchmark suite at scale factor 1 (approximately 6 million rows in `lineitem`, 1.5 million in `orders`, and proportional counts in the remaining six tables). The benchmark runs on PostgreSQL 17 on an Apple M3 system.

For each of the 22 TPC-H queries (excluding Q17, which did not terminate within a reasonable time for either configuration), we measure execution time under two configurations:

1. **Native:** the query runs against the base tables in the public schema with no branch layer active.
2. **Branch:** the session is switched to an empty child branch. The `lineitem` table resolves to the branch’s overlay view; all other tables resolve from public as usual.

The branch has an empty delta table, so we are measuring the pure structural overhead of the view-based overlay—the cost of the query planner incorporating the `LEFT JOIN` and `UNION ALL` into its plan—rather than the cost of replaying any actual deltas.

Each query is executed 3 times per configuration, alternating execution order between iterations (odd iterations run native first, even iterations run branch first) to neutralize cache warming bias. We report the median time for each configuration.

3.2 Results

Table 1 presents the benchmark results.

Query	Native (ms)	Branch (ms)	Overhead
Q1	1138.7	1330.0	1.17×
Q2	180.2	180.2	1.00×
Q3	260.1	446.1	1.72×
Q4	133.6	5156.1	38.60×
Q5	159.1	604.4	3.80×
Q6	222.3	226.2	1.02×
Q7	242.3	398.6	1.65×
Q8	221.6	501.1	2.26×
Q9	1002.4	1122.6	1.12×
Q10	376.9	426.6	1.13×
Q11	69.8	69.2	0.99×
Q12	360.4	340.9	0.95×
Q13	393.4	390.2	0.99×
Q14	220.3	226.0	1.03×
Q15	453.6	468.1	1.03×
Q16	129.0	127.0	0.98×
Q18	1443.1	2699.5	1.87×
Q19	316.3	318.7	1.01×
Q21	374.2	1659.7	4.44×
Q22	92.2	90.3	0.98×

Table 1. TPC-H query execution times (median of 3 runs, SF=1) comparing native PostgreSQL to the view-based branch overlay with an empty delta table.

Near-zero overhead (0.95–1.17×): Twelve of twenty-one queries—Q1, Q2, Q6, Q9–Q16, Q19, Q22—exhibit overhead at or below 1.17×. These queries primarily involve sequential scans, aggregations, and hash joins on `lineitem`. The LEFT JOIN anti-join structure allows PostgreSQL to use a Hash Anti Join plan, preserving parallel query execution. For several queries (Q11, Q12, Q13, Q16, Q22), the branch configuration is marginally *faster* than native, which we attribute to normal measurement variance.

Moderate overhead (1.65–2.26×): Queries Q3, Q7, Q8, and Q18 show moderate overhead. These queries involve multi-table joins where `lineitem` participates in hash or merge joins with other tables. The view structure introduces additional planning complexity and may prevent certain join reordering optimizations.

Severe overhead (3.80–38.60×): Queries Q4, Q5, and Q21 exhibit significant overhead. Q4 is the most extreme at 38.60×. Analysis of the query plans reveals the root cause: these queries use EXISTS correlated subqueries or semi-joins against `lineitem`. In the native case, PostgreSQL resolves these with a Nested Loop Semi Join using an index scan on `lineitem_pkey`—for Q4, this means approximately 56,000 efficient index probes. Through the view, the planner cannot push the correlated predicate through the UNION ALL to reach the underlying table’s index. Instead, it must materialize the entire view (scanning all 6 million rows), then join the result. This is a fundamental limitation of how PostgreSQL’s optimizer handles correlated subqueries through views containing UNION ALL.

3.3 Discussion

The results demonstrate that the view-based delta overlay is highly effective for queries that access `lineitem` through sequential scans, aggregations, and equality joins—the predominant access patterns in analytical workloads. The overhead for these queries is negligible, confirming that the LEFT JOIN anti-join pattern successfully preserves PostgreSQL’s parallel query capabilities.

The severe overhead on correlated subquery patterns (Q4, Q5, Q21) represents a fundamental trade-off of operating at the SQL view layer rather than at the storage layer. Systems like Neon, which implement branching below PostgreSQL’s storage engine at the page level, do not suffer from this limitation because the query planner is entirely unaware that branching exists—every branch appears as a fully independent database with its own indexes. However, such systems require replacing PostgreSQL’s entire storage layer and running distributed infrastructure (page servers, object storage), whereas our approach works as a lightweight extension installable with a single CREATE EXTENSION command.

A potential mitigation strategy is a *hybrid approach*: use the lightweight view overlay by default, but materialize a branch’s working copy when the delta table exceeds a size threshold or when query patterns requiring index access are detected. This would combine the instant creation and minimal storage of the view approach with the full query performance of a materialized copy, at the cost of additional implementation complexity.

4 Conclusion

We presented a branch-aware logical redo logging framework for PostgreSQL that enables Git-like branching semantics for relational tables. Through an iterative design process, we explored three architectures—temporary table materialization, eager materialized copies, and view-based delta overlay—each addressing limitations discovered in its predecessor.

Our final view-based design achieves its primary goals: branch creation is near-instant regardless of table size, storage is proportional only to the changes made on each branch, and users interact with branched tables using ordinary SQL with no special syntax. The key technical insight was that the choice of SQL anti-join pattern (LEFT JOIN vs. NOT IN) has a dramatic impact on whether PostgreSQL’s parallel query executor can engage, reducing overhead from 2.75× to 1.02× for typical analytical queries.

Evaluation on TPC-H at scale factor 1 shows that 12 of 21 queries run with overhead below 1.17×, confirming that the branch layer is nearly invisible for the majority of analytical workloads. A small number of queries involving correlated

subqueries exhibit higher overhead due to PostgreSQL’s inability to push predicates through UNION ALL views to underlying indexes—a fundamental optimizer limitation that affects any view-based approach.

Future work includes implementing a hybrid materialization strategy that could eliminate the correlated subquery overhead, adding branch diff operations to compare the state of two branches, and extending the system to support multi-table branching where a single branch operation spans multiple related tables.

5 Related Work

Dataset versioning is not a completely novel idea, but many known attempts have been at the storage or file layer rather than inside a relational engine.

5.1 MVCC

A semi-related problem that has been solved in the past is the challenge with concurrent updates to a row from user A and user B. Incorrect handling of this scenario can result in user B reading stale or only partially updated data. The solution proposed by MVCC [3] is to provide each user with their own isolated snapshot of the database at the given time of a transaction to preserve state across multiple users and link them together. This is a form of branching as multiple versions of a row are maintained, but this solution is only relevant for short term changes that interfere with concurrency.

MVCC therefore provides the primitives for multi-version storage, but does not go as far as to allow for persistent, long-term branching.

5.2 LakeFS

Data lake systems such as LakeFS [2] provide Git-like branching and snapshot semantics for large collections of files in object storage. LakeFS employs a copy-on-write model that maintains metadata to map snapshots to shared underlying files which prevents redundant duplication.

While more similar to our objective than MVCC, LakeFS operates distinctly at the file level; therefore, the system is incapable of versioning at the granularity of an individual tuple. Additionally, LakeFS operates outside the relational query engine, differing from our goal of embedding branching directly inside a relational engine.

5.3 OrpheusDB

The OrpheusDB paper [1] presents a well-motivated argument for version-controlled database systems, particularly in data science and analytical workflows where datasets frequently evolve while requiring reproducibility. OrpheusDB provides a solution similar to our goal by offering table level versioning granularity. However, similar to LakeFS, the system is implemented as a bolt-on layer rather than being embedded directly into the relational engine. More specifically,

versioning is managed through additional storage structures and metadata rather than by modifying the native visibility and storage mechanisms of database tuples.

References

- [1] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran. Orpheusdb: Bolt-on versioning for relational databases. *VLDB Endowment*, 2017.
- [2] A. Knüpfer and T. J. Callow. Research data version management and machine-actionable reproducibility for hpc. 2025.
- [3] D. R. K. Ports and K. Grittnner. Serializable snapshot isolation in postgresql. *VLDB Endowment*, 2012.